

Zero-Shot Chinese Character Recognition with Stroke-Level Decomposition

Jingye Chen, Bin Li*, Xiangyang Xue

Shanghai Key Laboratory of Intelligent Information Processing
School of Computer Science, Fudan University
{jingyechen19, libin, xyxue}@fudan.edu.cn

Abstract

Chinese character recognition has attracted much research interest due to its wide applications. Although it has been studied for many years, some issues in this field have not been completely resolved yet, *e.g.* the zero-shot problem. Previous character-based and radical-based methods have not fundamentally addressed the zero-shot problem since some characters or radicals in test sets may not appear in training sets under a data-hungry condition. Inspired by the fact that humans can generalize to know how to write characters unseen before if they have learned stroke orders of some characters, we propose a stroke-based method by decomposing each character into a sequence of strokes, which are the most basic units of Chinese characters. However, we observe that there is a one-to-many relationship between stroke sequences and Chinese characters. To tackle this challenge, we employ a matching-based strategy to transform the predicted stroke sequence to a specific character. We evaluate the proposed method on handwritten characters, printed artistic characters, and scene characters. The experimental results validate that the proposed method outperforms existing methods on both character zero-shot and radical zero-shot tasks. Moreover, the proposed method can be easily generalized to other languages whose characters can be decomposed into strokes.

1 Introduction

Chinese character recognition (CCR), which has been studied for many years, plays an essential role in many applications. Existing CCR methods usually rely on massive input data. For example, the HWDB1.0-1.1 database [Liu *et al.*, 2013] provides more than two million handwritten samples collected from 720 writers, including 3,866 classes overall. However, there are 70,244 Chinese characters in total according to the latest Chinese national standard GB18030-2005¹, thus collecting samples for each character is time-consuming.

*Corresponding author

¹https://zh.wikipedia.org/wiki/GB_18030

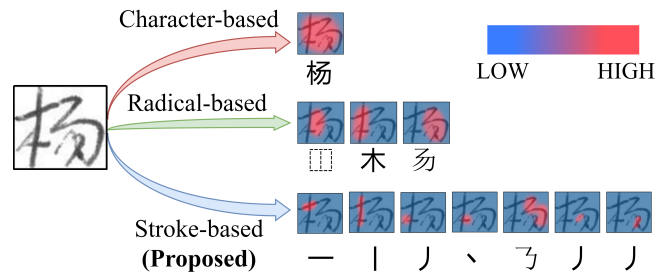


Figure 1: Three categories of CCR methods. The proposed method decomposes a character into a sequence of strokes, which are the smallest units of Chinese characters.

Early work on CCR mainly relies on hand-crafted features [Su and Wang, 2003; Shi *et al.*, 2003]. With the rapid development of deep learning, numerous CNN-based methods emerged and outperformed early traditional methods. Deep learning-based CCR methods can be divided into two categories: character-based methods and radical-based methods. Character-based methods treat each character as one class. For example, MCDNN [Ciresan and Meier, 2015] ensembles results of eight deep nets and reaches human-level performance. DirectMap [Zhang *et al.*, 2017] achieves new state-of-the-art in this competition by integrating the traditional directional map with a CNN model. However, these character-based methods can not recognize the characters that have not appeared in training sets, namely a *character zero-shot problem*. In this case, several radical-based methods are raised by treating each character as a radical sequence. DenseRAN [Wang *et al.*, 2018] takes the first attempt to see CCR as a tree-structured image captioning task. HDE [Cao *et al.*, 2020] designs a unique embedding vector for each Chinese character according to its radical-level constitution. However, there exist several drawbacks in existing radical-based methods: 1) Some radicals may not appear in training sets, namely a *radical zero-shot problem*. 2) Most of the previous radical-based methods ignore the fact that some characters have the same radical-level constitution. This problem even becomes worse as the increase of the alphabet capacity. 3) Since HDE [Cao *et al.*, 2020] functions in an embedding-matching manner, it needs to store the embeddings of *all* candidates in advance, which costs a lot of space. 4) The radical-level decomposition leads to a more severe class imbalance problem.

In this paper, inspired by the fact that humans can easily generalize to grasp how to write characters unseen before if they have learned stroke orders of some characters, we propose a stroke-based method by decomposing a character into a combination of five strokes, including *horizontal*, *vertical*, *left-falling*, *right-falling*, and *turning*. The five strokes all frequently appear in Chinese characters (in Figure 1, they present in one character simultaneously). Thus, there does not exist a *stroke zero-shot problem*. Furthermore, each character or radical is uniquely represented as a stroke sequence according to the Unicode Han Database², which helps pave the way for solving the character zero-shot and radical zero-shot problems fundamentally. However, we observe that there is a one-to-many relationship between stroke sequences and characters. To conquer this challenge, we employ a matching-based strategy to transform the predicted stroke sequence to a specific character in the test stage. The proposed method is validated on various kinds of datasets, including handwritten characters, printed artistic characters, and scene characters. The experimental results validate that the proposed method outperforms existing methods on both character zero-shot and radical zero-shot tasks. More interestingly, the proposed method can be easily generalized to those languages whose characters can be decomposed into strokes such as Korean. In summary, our contributions can be listed as follows:

- We propose a stroke-based method for CCR to fundamentally solve character and radical zero-shot problems.
- To tackle the one-to-many problem, we employ a matching-based strategy to transform the predicted stroke sequence to a specific character.
- Our method outperforms existing methods on both character zero-shot and radical zero-shot tasks, and can be generalized to other languages whose characters can be decomposed into strokes.

2 Related Work

2.1 Character-based Approaches

Traditional Methods. Traditional character-based methods use hand-crafted features like Gabor features [Su and Wang, 2003], directional features [Jin *et al.*, 2001], and vector features [Chang, 2006]. However, the performance of them are limited by these low-capacity features [Chen *et al.*, 2020].

Deep Learning-based Methods. With the development of deep learning, several methods employ CNN-based models, which can automatically extract features from given images. MCDNN [Cireřan and Meier, 2015] is the first case to employ CNN for CCR by ensembling eight models while outperforming human-level performance on recognizing handwritten characters. After that, ART-CNN [Wu *et al.*, 2014] alternatively trains a relaxation CNN and takes the first place in the ICDAR2013 competition [Yin *et al.*, 2013]. In [Xiao *et al.*, 2019], a template-instance loss is employed to rebalance easy and difficult Chinese instances. However, these methods rely on massive data and can not handle characters that have not appeared in training sets.

²<https://www.zdic.net/> collects the stroke orders for the majority of Chinese characters from Unicode Han Database.

2.2 Radical-based Approaches

Traditional Methods. Before the deep learning era, several radical-based methods are proposed using traditional strategies. In [Wang *et al.*, 1996], a recursive hierarchical scheme is introduced for radical extraction of Chinese characters. It needs accurate pre-extracted strokes, which is difficult to obtain due to the scribbled written styles in datasets. In [Shi *et al.*, 2003], a method based on active radical modeling is proposed. It omits the stroke extraction procedure and achieves higher recognition accuracy. However, the pixel-wise matching and shape-parameter searching are time-consuming.

Deep Learning-based Methods. In recent years, the development of deep learning helps pave the way for radical-based methods. DenseRAN [Wang *et al.*, 2018] treats the recognition task as image captioning by regarding each character as a radical sequence. Based on DenseRAN, STN-DenseRAN [Wu *et al.*, 2019] employs a rectification block for distorted character images. FewShotRAN [Wang *et al.*, 2019] maps each radical to a latent space and constrains features of the same class to be close. Recently, HDE [Cao *et al.*, 2020] designs an embedding vector for each character using radical-composition knowledge and learns the transformation from the sample space to the embedding space. These methods are capable of tackling the character zero-shot problem. However, some radicals may not appear in training sets in a data-hungry condition, which leads to another dilemma called radical zero-shot. Hence, these radical-based methods have not solved the zero-shot problem fundamentally.

2.3 Stroke-based Approaches

Existing stroke-based methods usually rely on traditional strategies. In [Kim *et al.*, 1999a], the authors propose a stroke-guided pixel matching method, which can tolerate mistakes caused by stroke extraction. In [Kim *et al.*, 1999b], a method based on mathematical morphology is raised to decompose Chinese characters. A model-based structural matching method [Liu *et al.*, 2001] is proposed with each character described by an attributed relational graph. In [Su and Wang, 2003], they present a method based on a directional filtering technique. These traditional methods need hand-crafted features, which are hard to adapt to different fields and applications. In general, these works have inspired us to combine stroke knowledge with deep learning models.

3 Preliminary Knowledge of Strokes

Strokes are the smallest units for each Chinese character. When humans start learning Chinese, they usually learn to write strokes at first, then radicals, and finally the whole characters. Moreover, there exists a *regular pattern* in Chinese stroke orders, usually following left to right, top to bottom, and outside in. In other words, when humans have learned the stroke orders of some characters, they can naturally generalize to grasp how to write other characters (even though humans have never seen them before), which inspires us to design a stroke-based model to tackle the zero-shot problem.

According to the Chinese national standard GB18030-2005, five basic strokes are *horizontal*, *vertical*, *left-falling*,

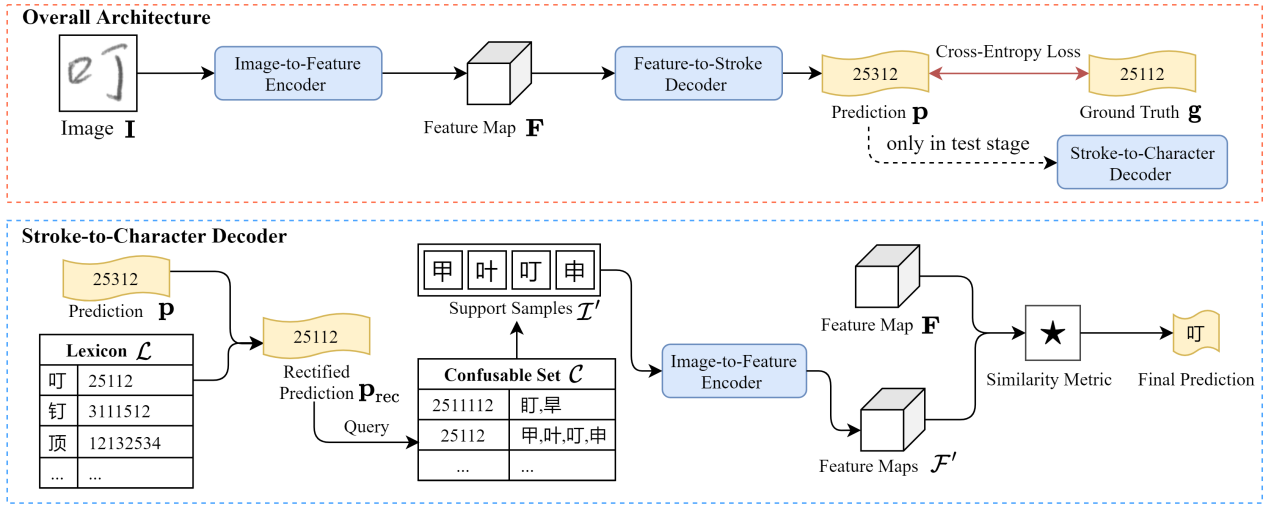


Figure 2: The overall architecture of the proposed model involves one encoder and two decoders at different levels. The feature-to-stroke decoder is used when training, whereas the stroke-to-character decoder is utilized when testing. Five strokes are encoded from “1” to “5”.

Category	(1) Horizontal	(2) Vertical	(3) Left-falling	(4) Right-falling	(5) Turning
Instances					

Figure 3: Five basic categories of strokes. There are several instances of various shapes in each basic category.

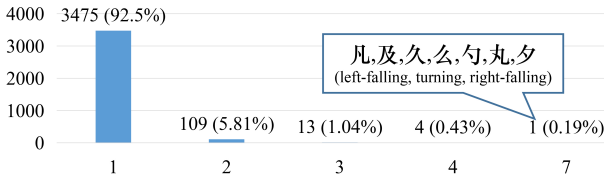


Figure 4: Illustration of the one-to-many problem. The x -axis denotes the one-to- n stroke and the y -axis denotes the quantity.

right-falling, and *turning*. As shown in Figure 3, each category contains instances of different shapes. Please note that the turning category contains more kinds of instances and we only show five of them in Figure 3. Stroke orders for each character are collected from the Unicode Han Database.

In fact, we observe that there is a one-to-many relationship between stroke sequences and characters. As shown in Figure 4, we explore the distribution of one-to- n sequences in 3,755 Level-1 characters (most commonly-used). Most of the sequences (about 92.5%) can perfectly match a single character. In the worst case, a sequence can correspond to seven possible results ($n = 7$). Therefore, it is necessary to design a module to match each sequence with a specific character.

4 Methodology

The overall architecture is shown in Figure 2. In the training stage, the input image is fed into an encoder-decoder architecture to generate a stroke sequence. In the test stage, the

sequence is first rectified by a stroke-sequence lexicon, which is further sent to a Siamese architecture to match a character from a confusable set. Details are introduced in the following.

4.1 Image-to-Feature Encoder

In recent years, ResNet [He *et al.*, 2016] plays a significant role in optical character recognition tasks [Wang *et al.*, 2019]. Residual blocks relieve the gradient vanishing problem, thus enabling a deeper network to fit training data more efficiently. We employ building blocks [He *et al.*, 2016] containing two successive 3×3 CNN as the unit of ResNet. Details of the encoder are shown in Supplementary Materials. For a given three-channel image $\mathbf{I} \in H \times W \times 3$, the encoder outputs a feature map $\mathbf{F}$ of size $\frac{H}{2} \times \frac{W}{2} \times 512$ for further decoding.

4.2 Feature-to-Stroke Decoder

We employ the basic design of Transformer decoder [Vaswani *et al.*, 2017]. The architecture is shown in Supplementary Materials. We denote the ground truth as $\mathbf{g} = (g_1, g_2, \dots, g_T)$. A cross-entropy loss is employed to optimize the model: $l = -\sum_{t=1}^T \log p(g_t)$, where T is the length of the sequential label and $p(g_t)$ is the probability of class g_t at the time step t .

4.3 Stroke-to-Character Decoder

Since a stroke sequence may not match with a specific character, a stroke-to-character decoder is further proposed in the test stage. Firstly, we build a lexicon $\mathcal{L}$ that contains stroke sequences of *all characters*. Nevertheless, in the worst case, the predicted sequence $\mathbf{p}$ may fail to match any characters in the lexicon, *i.e.* $\mathbf{p} \notin \mathcal{L}$. So we choose the one that has the least edit distance with the prediction $\mathbf{p}$ as the rectified prediction $\mathbf{p}_{\text{rec}}$. If matched initially, we consider the rectified prediction to be just the same as the original one, *i.e.* $\mathbf{p}_{\text{rec}} = \mathbf{p}$.

As shown in Figure 2, we manually collect a dictionary called *confusable set* $\mathcal{C}$ containing those one-to-many stroke sequences, *i.e.* those one-to-one characters will not appear in $\mathcal{C}$. If the rectified prediction is not in this set, *i.e.* $\mathbf{p}_{\text{rec}} \notin \mathcal{C}$, the

decoder will generate the corresponding character directly. Otherwise, we employ a matching-based strategy via comparing features between the source image $\mathbf{I}$ and support samples $\mathcal{I}'$ using a Siamese architecture. Specifically, for one head in the Siamese architecture, the feature map $\mathbf{F}$ of the input image $\mathbf{I}$ is given. For the other head, several support samples $\mathcal{I}'$ containing printed images of characters with the same stroke sequence are fed to the encoder to generate a list of feature maps $\mathcal{F}' = \{\mathbf{F}'_1, \mathbf{F}'_2, \dots, \mathbf{F}'_N\}$, where N is the number of possible results. We calculate similarity scores between $\mathbf{F}$ and each feature map $\mathbf{F}'_i$, then selecting the one which is the most similar to $\mathbf{F}$ as the final result:

$$i^* = \arg \max_{i \in \{1, 2, \dots, N\}} D(\mathbf{F}, \mathbf{F}'_i) \quad (1)$$

where i^* is the index of result and D is the similarity metric:

$$D(x_1, x_2) = \begin{cases} 1 - \|x_1 - x_2\|_2 & \text{Euclidean metric} \\ \frac{x_1^T x_2}{\|x_1\| \times \|x_2\|} & \text{Cosine metric} \end{cases} \quad (2)$$

Different from FewShotRAN [Wang *et al.*, 2019], we do not use support samples during training, which guarantees the principle of zero-shot learning. Compared with HDE [Cao *et al.*, 2020], our method costs less space since we only need to store the features of confusable characters in advance.

5 Experiments

In this section, we first introduce the basic setting of our experiments, and analyze two similarity metrics. Then we compare the proposed method with existing zero-shot methods on various datasets. Finally, we have a discussion on our method in terms of generalization ability and time efficiency.

Datasets. The datasets used in our experiments are introduced in the following and examples are shown in Figure 5.

- **HWDB1.0-1.1** [Liu *et al.*, 2013] contains 2,678,424 offline handwritten Chinese character images. It contains 3,881 classes collected from 720 writers.
- **ICDAR2013** [Yin *et al.*, 2013] contains 224,419 offline handwritten Chinese character images with 3,755 classes collected from 60 writers.
- **Printed artistic characters** We collect 105 printed artistic fonts for 3,755 characters (394,275 samples). Each image is of size 32×32 and the font size is 32px.
- **CTW** [Yuan *et al.*, 2019] contains Chinese characters collected from street views. The dataset is challenging due to its complexity in backgrounds, font types, etc. It contains 812,872 samples (760,107 for training and 52,765 for testing) with 3,650 classes.
- **Support samples** The size of support samples is 32×32 with the font size 32px. Two widely-used fonts including Simsun and Simfang (not in the artistic fonts) are used. We take the average of similarity scores between features of input and two support samples when testing.

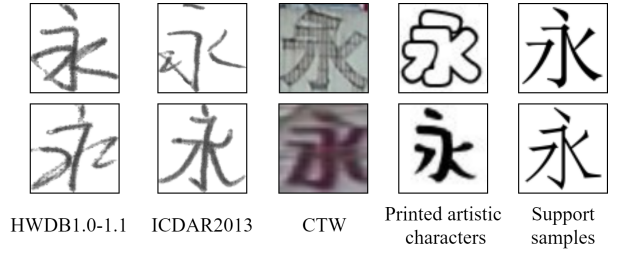


Figure 5: Examples of each dataset of the Chinese character “Yong”.

Evaluation Metric. Character Accuracy (CACC) is used as the evaluation metric since a stroke sequence may not match a specific character. We follow the *traditional way* to construct the candidate set by combining categories appearing in both training sets and test sets [Wang *et al.*, 2018; Wu *et al.*, 2019].

Implementation Details. We implement our method with PyTorch and conduct experiments on an NVIDIA RTX 2080Ti GPU with 11GB memory. The Adadelat optimizer is used with the learning rate set to 1. The batch size is set to 32. Each input image is resized to 32×32 and normalized to $[-1, 1]$. We adopt a weight decay rate of 10^{-4} in zero-shot settings to avoid overfitting. For the seen character setting, we ensemble the results of ten models. No pre-training and data augmentation strategies are used in our experiments.

5.1 Choice of Similarity Metrics

In the test stage, we evaluate the performance with Euclidean metric and Cosine metric using the same model, with the parameters fixed after training. We choose samples with labels in the first 1500 classes in 3,755 Level-1 commonly-used characters from HWDB1.0-1.1 as the training set, while picking out images from ICDAR2013 as the test set, whose labels are both in the last 1000 classes and in the confusable set, for validation. The experimental results show that the performance of Cosine similarity achieves 90.56%, which is better than the Euclidean metric (89.39%). We suppose that the Cosine metric cares most about whether a specific feature exists, while caring less about its exact value, which is more suitable for characters with various styles. Hence, we use the Cosine similarity metric in the following experiments.

5.2 Experiments on Handwritten Characters

We conduct experiments on handwritten characters in three settings, including character zero-shot, radical zero-shot, and seen character. 3,755 Level-1 commonly-used characters are used as candidates during testing.

Experiments in Character Zero-shot Setting

From HWDB1.0-1.1, we choose samples with labels in the first m classes of 3,755 characters as the training set, where m ranges in $\{500, 1000, 1500, 2000, 2755\}$. From ICDAR2013, we choose samples with labels in the last 1000 classes as the test set. As shown in the top-left of Table 1, our method outperforms other methods with various amounts of training classes. For a fair comparison, we additionally experiment with the same encoder (DenseNet) and decoder (RNN) that used in DenseRAN [Wang *et al.*, 2018] and the results show

Handwritten	m for Character Zero-Shot Setting					Handwritten	n for Radical Zero-Shot Setting				
	500	1000	1500	2000	2755		50	40	30	20	10
DenseRAN [2018]	1.70%	8.44%	14.71%	19.51%	30.68%	DenseRAN [2018]	0.21%	0.29%	0.25%	0.42%	0.69%
HDE ³ [2020]	4.90%	12.77%	19.25%	25.13%	33.49%	HDE ³ [2020]	3.26%	4.29%	6.33%	7.64%	9.33%
Ours	5.60%	13.85%	22.88%	25.73%	37.91%	Ours	5.28%	6.87%	9.02%	14.67%	15.83%

Printed Artistic	m for Character Zero-Shot Setting					Printed Artistic	n for Radical Zero-Shot Setting				
	500	1000	1500	2000	2755		50	40	30	20	10
DenseRAN [2018]	0.20%	2.26%	7.89%	10.86%	24.80%	DenseRAN [2018]	0.07%	0.16%	0.25%	0.78%	1.15%
HDE ³ [2020]	7.48%	21.13%	31.75%	40.43%	51.41%	HDE ³ [2020]	4.85%	6.27%	10.02%	12.75%	15.25%
Ours	7.03%	26.22%	48.42%	54.86%	65.44%	Ours	11.66%	17.23%	20.62%	31.10%	35.81%

Scene	m for Character Zero-Shot Setting					Scene	n for Radical Zero-Shot Setting				
	500	1000	1500	2000	3150		50	40	30	20	10
DenseRAN [2018]	0.15%	0.54%	1.60%	1.95%	5.39%	DenseRAN [2018]	0%	0%	0%	0%	0.04%
HDE ³ [2020]	0.82%	2.11%	3.11%	6.96%	7.75%	HDE ³ [2020]	0.18%	0.27%	0.61%	0.63%	0.90%
Ours	1.54%	2.54%	4.32%	6.82%	8.61%	Ours	0.66%	0.75%	0.81%	0.94%	2.25%

Table 1: Results of character zero-shot (left column) and radical zero-shot (right column) tasks on handwritten characters (top row), printed artistic characters (middle row), and scene characters (bottom row). We omit character-based methods since they can not tackle these tasks.

that our method is still better than the compared methods (see Supplementary Materials). The reasons that our method performs better can result from two aspects: 1) The frequency of each stroke in the training set is far more than each radical, and therefore helps the model converge better. 2) Our method alleviates the class imbalance problem, which commonly exists in the character-based and radical-based methods (see more details in Supplementary Materials).

Experiments in Radical Zero-shot Setting

The setup follows three steps: 1) Calculate the frequency of each radical in 3,755 Level-1 commonly-used characters. 2) If a character contains a radical that appears less than n times ($n \in \{50, 40, 30, 20, 10\}$), move it into set $\mathcal{S}_{\text{TEST}}$; otherwise, move it into $\mathcal{S}_{\text{TRAIN}}$. 3) Select all samples with labels in $\mathcal{S}_{\text{TRAIN}}$ from HWDB1.0-1.1 as the training set, and all samples with labels in $\mathcal{S}_{\text{TEST}}$ from ICDAR2013 as the test set. In this manner, the characters in the test set contain so-called unseen radicals. The training class increases when n decreases (see Supplementary Materials). As shown in the top-right of Table 1, our method outperforms all the compared methods since all the strokes have been supervised in training sets. Moreover, the proposed method can infer the stroke orders of unseen radicals based on the knowledge learned from training sets. Although DenseRAN [Wang *et al.*, 2018] and HDE [Cao *et al.*, 2020] can alleviate this problem using lexicon rectification and embedding matching, they are still subpar due to the lack of direct supervisions on unseen radicals.

Experiments in Seen Character Setting

Different from zero-shot settings, the seen character setting represents utilizing samples whose labels have appeared in training sets for testing. We use the full sets of HWDB1.0-1.1 for training and ICDAR2013 for testing. Firstly, we evaluate the performance of those samples whose labels are not in the confusable set $\mathcal{C}$. As shown in Supplementary Materials, our method yields better performance compared with two encoder-decoder architectures at different levels due to more occurrences for each category. The experimental results on the full set of ICDAR2013 are shown in Table 2. Our method

Method	CACC
Human Performance [Yin <i>et al.</i> , 2013]	96.13%
HCCR-GoogleNet [Zhong <i>et al.</i> , 2015]	96.35%
DirectMap+ConvNet+Adaptation [Zhang <i>et al.</i> , 2017]	97.37%
M-RBC+IR [Yang <i>et al.</i> , 2017]	97.37%
DenseRAN [Wang <i>et al.</i> , 2018]	96.66%
FewShotRAN [Wang <i>et al.</i> , 2019]	96.97%
HDE [Cao <i>et al.</i> , 2020]	97.14%
Template+Instance [Xiao <i>et al.</i> , 2019]	97.45%
Ours	96.28%
Ours + Character-based	96.74%

Table 2: The results in seen character setting on ICDAR2013.

does not achieve ideal performance since the matching-based strategy follows an unsupervised manner. We additionally train a character-based model (only need to modify the last linear layer of the proposed model) on the same training set to resolve this problem. If the feature-to-stroke decoder outputs a stroke sequence that belongs to $\mathcal{C}$, we employ this model to generate the final prediction as a alternative, which boosts CACC by 0.46%. It can be treated as combining advantages of two models at different levels. See visualizations in Figure 6 and more analyses in Supplementary Materials.

5.3 Experiments on Printed Artistic Characters

We conduct experiments with printed artistic characters in two settings, including character zero-shot and radical zero-shot (see the middle row of Table 1). The division of this dataset is shown in Supplementary Materials. The performance of our method is more than twice as HDE [Cao *et al.*, 2020] in the radical zero-shot setting. Compared with handwritten characters, the printed ones have relatively clearer strokes, which benefits the proposed stroke-based method.

³We reimplement the method in zero-shot settings using the authors' provided code. Please note that we build up candidate sets following the traditional way which differs from the way in [Cao *et al.*, 2020]. We set the input size to 32×32 for a fair comparison.

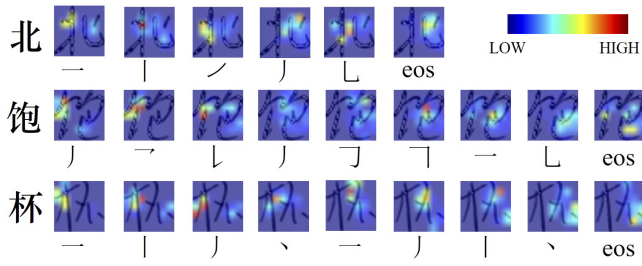


Figure 6: Visualization of attention maps for three characters. “eos” is the stop symbol denoting “end of sequence”.

Method	CACC
ResNet50 [He <i>et al.</i> , 2016]	79.46%
ResNet152 [He <i>et al.</i> , 2016]	80.94%
DenseNet [Huang <i>et al.</i> , 2017]	79.88%
DenseRAN [Wang <i>et al.</i> , 2018]	85.56%
FewshotRAN [Wang <i>et al.</i> , 2019]	86.78%
HDE [Cao <i>et al.</i> , 2020]	89.25%
Ours	85.29%
Ours + Character-based	85.90%

Table 3: The results in seen character setting on CTW.

5.4 Experiments on Scene Characters

The division of CTW [Yuan *et al.*, 2019] is shown in Supplementary Materials. As shown in the bottom row of Table 1, the proposed method outperforms the compared methods in the character zero-shot and radical zero-shot settings in most cases. However, all of these methods still can not achieve human-level performance in zero-shot tasks, as the recognition on scene characters faces many challenges such as low resolution and complicated backgrounds. The experimental results in seen character setting are shown in Table 3.

5.5 Discussion

Generalize to New Languages

To validate the ability of cross-language generalization, we test our method on Korean characters after training using the full set of printed artistic characters. Like Chinese characters, each Korean character can also be uniquely decomposed into sequences with five strokes. The stroke orders of each character are collected from Wikipedia⁴. Moreover, we observe that the written pattern is nearly the same as Chinese characters. Two samples are shown in Figure 7. To construct the test set, we manually collect 119 fonts for 577 Korean characters. The experimental results show that the model trained on Chinese datasets can achieve accuracy of 17.50% on the Korean dataset although suffering from domain gaps such as different font styles and distributions of five strokes. We consider that the stroke knowledge bridges the gap between different languages, thus making it possible to build up a cross-language recognizer while only need to replace candidate classes and support samples. Moreover, previous methods do not have this ability as different languages usually have different character- and radical-level representations.

⁴<https://en.wikipedia.org/wiki/%E3%85%92>

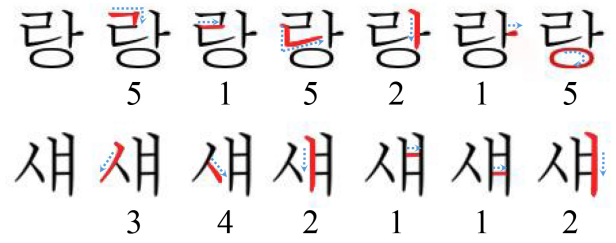


Figure 7: Korean characters and the corresponding stroke sequences.

Furthermore, we conduct a character zero-shot experiment on the Korean dataset. We randomly shuffle the 577 characters five times and utilize the first $m\%$ characters for training and the other part for testing, where $m \in \{20, 50, 80\}$. The performance reaches 23.09%, 65.94%, and 78.00% when training using three proportions of datasets. When trained on 80% of the dataset, the model is capable of recognizing over three-quarters of unseen characters, which further validates that the stroke knowledge can work on Korean characters.

Time Efficiency

We investigate the time cost of encoder-decoder architectures at three levels (refer to Figure 1). For a fair comparison, we employ the same ResNet-Transformer architecture. The batch size is set to 32 and we take the average time of 100 iterations. The character-based method (0.24s) outperforms all other methods since it does not need to predict recurrently during testing, *i.e.* only need to decode once. Although our stroke-based method (1.24s) has to conduct decoding at two different levels, it is only a six-category classification task (includes one stop symbol) in the feature-to-stroke decoder, thus it has fewer parameters in the last fully connected layer and performs faster than the radical-based method (1.38s).

6 Conclusions

In this paper, we propose a stroke-based method to deal with the zero-shot Chinese character recognition problems inspired by the generalization ability of human beings with stroke knowledge. Furthermore, we put forward a matching-based strategy to tackle the one-to-many challenge benefiting from the Siamese architecture. The experimental results validate that our method outperforms other existing methods in both character zero-shot and radical zero-shot settings on various kinds of datasets. Meanwhile, our method can be easily generalized to new languages, which further verifies that the stroke knowledge enables building up cross-language recognizers for a bundle of East Asian characters.

Acknowledgements

This research was supported in part by STCSM Projects (20511100400, 20511102702), Shanghai Municipal Science and Technology Major Projects (2018SHZDZX01, 2021SHZDZX0103), Shanghai Research and Innovation Functional Program (17DZ2260900), the Program for Professor of Special Appointment (Eastern Scholar) at Shanghai Institutions of Higher Learning, and ZJLab.

References

- [Cao *et al.*, 2020] Zhong Cao, Jiang Lu, Sen Cui, and Changshui Zhang. Zero-shot handwritten chinese character recognition with hierarchical decomposition embedding. *PR*, 107:107–488, 2020.
- [Chang, 2006] Fu Chang. Techniques for solving the large-scale classification problem in chinese handwriting recognition. In *SACH*, pages 161–169, 2006.
- [Chen *et al.*, 2020] Xiaoxue Chen, Lianwen Jin, Yuanzhi Zhu, Canjie Luo, and Tianwei Wang. Text recognition in the wild: A survey. *ACMCS*, 2020.
- [Cireřan and Meier, 2015] Dan Cireřan and Ueli Meier. Multi-column deep neural networks for offline handwritten chinese character classification. In *IJCNN*, pages 1–6, 2015.
- [He *et al.*, 2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016.
- [Huang *et al.*, 2017] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *CVPR*, pages 4700–4708, 2017.
- [Jin *et al.*, 2001] Lian-Wen Jin, Jun-Xun Yin, Xue Gao, and Jiang-Cheng Huang. Study of several directional feature extraction methods with local elastic meshing technology for hccr. In *ICYCS*, pages 232–236, 2001.
- [Kim *et al.*, 1999a] In-Jung Kim, Cheng-Lin Liu, and Jin-Hyung Kim. Stroke-guided pixel matching for handwritten chinese character recognition. In *ICDAR*, pages 665–668, 1999.
- [Kim *et al.*, 1999b] Jin Wook Kim, Kwang In Kim, Bong Joon Choi, and Hang Joon Kim. Decomposition of chinese character into strokes using mathematical morphology. *PRL*, 20(3):285–292, 1999.
- [Liu *et al.*, 2001] Cheng-Lin Liu, In-Jung Kim, and Jin H Kim. Model-based stroke extraction and matching for handwritten chinese character recognition. *PR*, 34(12):2339–2352, 2001.
- [Liu *et al.*, 2013] Cheng-Lin Liu, Fei Yin, Da-Han Wang, and Qiu-Feng Wang. Online and offline handwritten chinese character recognition: benchmarking on new databases. *PR*, 46(1):155–162, 2013.
- [Shi *et al.*, 2003] Daming Shi, Steve R. Gunn, and Robert I. Damper. Handwritten chinese radical recognition using nonlinear active shape models. *TPAMI*, 25(2):277–280, 2003.
- [Su and Wang, 2003] Yih-Ming Su and Jhing-Fa Wang. A novel stroke extraction method for chinese characters using gabor filters. *PR*, 36(3):635–647, 2003.
- [Vaswani *et al.*, 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, pages 5998–6008, 2017.
- [Wang *et al.*, 1996] An-Bang Wang, Kuo-Chin Fan, and Wei-Hsien Wu. A recursive hierarchical scheme for radical extraction of handwritten chinese characters. In *ICPR*, pages 240–244, 1996.
- [Wang *et al.*, 2018] Wenchao Wang, Jianshu Zhang, Jun Du, Zi-Rui Wang, and Yixing Zhu. Densaran for offline handwritten chinese character recognition. In *ICFHR*, pages 104–109, 2018.
- [Wang *et al.*, 2019] Tianwei Wang, Zecheng Xie, Zhe Li, Lianwen Jin, and Xiangle Chen. Radical aggregation network for few-shot offline handwritten chinese character recognition. *PRL*, 125:821–827, 2019.
- [Wu *et al.*, 2014] Chunpeng Wu, Wei Fan, Yuan He, Jun Sun, and Satoshi Naoi. Handwritten character recognition by alternately trained relaxation convolutional neural network. In *ICFHR*, pages 291–296, 2014.
- [Wu *et al.*, 2019] Changjie Wu, Zi-Rui Wang, Jun Du, Jianshu Zhang, and Jiaming Wang. Joint spatial and radical analysis network for distorted chinese character recognition. In *ICDARW*, pages 122–127, 2019.
- [Xiao *et al.*, 2019] Yao Xiao, Dan Meng, Cewu Lu, and Chi-Keung Tang. Template-instance loss for offline handwritten chinese character recognition. In *ICDAR*, pages 315–322, 2019.
- [Yang *et al.*, 2017] Xiao Yang, Dafang He, Zihan Zhou, Daniel Kifer, and C Lee Giles. Improving offline handwritten chinese character recognition by iterative refinement. In *ICDAR*, pages 5–10, 2017.
- [Yin *et al.*, 2013] Fei Yin, Qiu-Feng Wang, Xu-Yao Zhang, and Cheng-Lin Liu. Icdar 2013 chinese handwriting recognition competition. In *ICDAR*, pages 1464–1470, 2013.
- [Yuan *et al.*, 2019] Tai-Ling Yuan, Zhe Zhu, Kun Xu, Cheng-Jun Li, Tai-Jiang Mu, and Shi-Min Hu. A large chinese text dataset in the wild. *JCST*, 34(3):509–521, 2019.
- [Zhang *et al.*, 2017] Xu-Yao Zhang, Yoshua Bengio, and Cheng-Lin Liu. Online and offline handwritten chinese character recognition: A comprehensive study and new benchmark. *PR*, 61:348–360, 2017.
- [Zhong *et al.*, 2015] Zhuoyao Zhong, Lianwen Jin, and Zecheng Xie. High performance offline handwritten chinese character recognition using googlenet and directional feature maps. In *ICDAR*, pages 846–850, 2015.